Ultra runs: ultracode and ultrareview, spent well

Claude Code has two "go big" modes, and they're easy to mix up.

Kind Pattern | Changed 2026-11-12 | For Anyone with three free runs to spend

You are here: drasticstatic lab › `my-template` › `walkthroughs` › `ULTRA-RUNS.md` · public · `main` is the working branch

TL;DR

1. **Ultracode** makes your helper run *every* substantive task as a team of agents. It's a switch, it uses your plan's usage, and it has no free runs.
2. **Ultrareview** sends a team of reviewers into the cloud to find and double-check bugs. It never edits code. Pro and Max accounts get **three free runs, once, ever**.
3. **The trap:** ultrareview reviews the *difference* between your branch and `main`. If all the work is already on `main`, that difference is empty and the free run is gone. Stage a review first (below).
4. **One coordinator, many workers:** set the model and effort once, at the top; hand bulk work to subagents on a model of their own (below).

> ***Handing this to your AI helper?*** *Say: "Read ULTRA-RUNS.md. Help me stage a review for and tell me when it's ready. Don't start the review yourself." (Helpers can't start an ultrareview; you do.)*

Ultracode

Turn on	`/effort ultracode` (or in the `/effort` slider, press `Tab` to flip **Ultracode**, then `Enter`)
Turn off	`/effort ultracode off`
Effort	Works at whatever effort is set. Set it first (e.g. `/effort high`). `claude --effort ultracode` also sets `xhigh`.
Cost	Draws on your plan's usage limits; you'll reach session and weekly limits sooner.
Good for	Deep reconnaissance across many repos, audits, migrations, big polish passes.

Before you turn it on:

1. Everything committed and pushed; every repo you'll touch is synced.
2. The repo has a `CLAUDE.md` with its rules. Every agent the run starts reads it, so this is where the rules that must never break go.
3. A written brief: what to look at, what "done" means, what's out of scope.
4. Connected tools (MCP servers, the browser extension) signed in **before** the run, not during it.
5. Approve edits by hand until you're ready, so the switch doesn't flip early.

Who publishes: the main session, not the agents

Publishing and repository creation run in the main session; subagents prepare. In a multi-agent run, the safety classifier refuses `gh repo create` (and can refuse a first push to a new public repository) when a background agent attempts it, even where

`~/.claude/settings.json` allows the command. Plan for it:

Agents do	The main session does, with the person approving each prompt
write, commit and push to repositories that already exist; open pull requests	`gh repo create`, a repository's first public push, enabling Pages
stage a new repository in a scratch folder, with its commits and a `CREATE-AND-PUSH.sh`	run that script, then move the clone to its home
list the exact commands they could not run (`main_session_steps`)	run them in small batches and read each result back
prepare deploy-key and ruleset commands	generate keys, add deploy keys, set secrets, apply rulesets

A run that ignores this ends with work that is staged but not live, and a list of refusals to replay by hand. Write the brief so each agent returns its unfinished steps as exact commands.

Ultrareview

Start	`/code-review ultra` (your branch vs `main`) or `/code-review ultra <PR number>` (a GitHub pull request)
Time	About 15–25 minutes, in the cloud
Free runs	Three, one-time per account. **A run you stop early, or that fails, still uses one.**
Output	Verified findings in your session. It doesn't change code. Add `--post` on a PR to post them there.

Stage a review so the run sees the code

When the work is already on `main`, give the review a base that doesn't have the code yet:

```
git switch -c review/code-baseline main
git rm -r -q <the code folders to review>              # e.g. src/ scripts/ tests/ .github/ (main brings them back)
git commit -m "review baseline: main without the code under review (never merge)"
git push -u origin review/code-baseline
git switch main
gh pr create --draft --base review/code-baseline --head main \
  --title "Ultrareview target: <what> (do not merge)" --body "Review-only PR. Diff = the code under review."
gh pr diff <number> --name-only | wc -l                 # not zero, and only the files you meant
```

Then `/code-review ultra <number>`. The PR follows `main`, so it stays a valid review target as work lands. Keep lockfiles, images and generated files in the baseline so the reviewers spend their attention on code.

Choosing what to spend the three on

Spend them on code that is **large, live, and hard to test by hand**: a site people use, contract or payment code, CI that publishes. A good order is: recon first (what is where, and what's risky), then stage a PR per target, then run the reviews one at a time and fix each one's findings before the next.

Model and effort: one coordinator, many workers

In short: pick the model and the effort at the top of a session and leave them; let the session hand work to helpers (subagents) that run on a model of their own. Each claim below was checked against Anthropic's Claude Code docs on 2026-11-12 (sources at the end of this section); the two marked *fleet practice* are ours, not Anthropic's.

Effort levels, and what each is for

Level	Anthropic's description
`low`	quick exchanges where you review each result: brainstorming, a first sketch, a rename
`medium`	day-to-day work with a clear scope, such as a new feature. The default on Opus 5.5, Sonnet 5.5, Haiku 5.5 and Fable 5.2
`high`	work where verification matters or edge cases are likely, such as fixing a bug in existing code
`xhigh`	deeper reasoning at a higher token spend
`max`	hard problems Claude works through without you. The docs warn it "may show diminishing returns and is prone to overthinking", and it applies to the current session only
`ultracode`	**not a level:** a setting that plans a workflow (a team of agents) for each substantive task, at whatever level is set

Which levels exist depends on the model; a level the model lacks falls back to the highest one below it. Set it with `/effort <level>`, the slider in `/model`, `claude --effort <level>`, the `CLAUDE_CODE_EFFORT_LEVEL` environment variable (which wins over all the others), or `effort` in a skill's or subagent's frontmatter.

One mind for the foundation, a team for the sweep

- *Fleet practice.* **A planning or foundation session is one mind.** When nothing exists yet, the work is a chain of decisions that each depend on the last, so the fleet runs it as one session at a higher effort, with no fan-out. Anthropic's own page is more open than we are here: it lists "a hard plan worth drafting from several independent angles" as a fit for a workflow.
- **Fan-out pays once there is a surface to work over.** Anthropic's examples for workflows are a codebase-wide bug sweep, a 500-file migration, research whose sources must be cross-checked, and an audit of every route for the same flaw. With ultracode on, "each request uses more tokens and takes longer", so turn it on when the code to audit or sweep already exists (*fleet practice*, matching that advice).

A coordinator on one model, workers on another

A session on one model can start subagents on a different one. Claude Code picks a subagent's model in this order:

1. the `model` passed on that **Agent tool call** (an alias only: `sonnet`, `opus`, `haiku` or `fable`);
2. the `model` in the **subagent's definition** (`.claude/agents/<name>.md` frontmatter: an alias, a full model ID, or `inherit` for the main conversation's model);
3. the **`CLAUDE_CODE_SUBAGENT_MODEL`** environment variable, a default for subagents that set nothing else (`CLAUDE_CODE_SUBAGENT_MODEL_FORCE=1` makes it apply to every subagent, ignoring the two above);
4. the main conversation's model.

Effort follows the model. A subagent on the session's own model runs at the session's level; on another model that level does not apply, so give the definition its own `effort`. A `/model` switch reaches subagents that inherit, from the next one started. The `opusplan` alias is the same split inside one session: Opus in plan mode, Sonnet to carry it out.

What a change costs mid-session

Change	What the docs say happens

Model (`/model`)	each model has its own cache, so the next request reads the whole conversation with no cache hits. While the cache is warm, Claude Code asks you to confirm. Each `opusplan` toggle into or out of plan mode is a model switch too
Effort	on most models the same full uncached re-read. **With an API key or a Claude subscription, the cache stays intact on Opus 5.5, Sonnet 5.5, Haiku 5.5 and Fable 5.1 and later.** On Amazon Bedrock, Google Cloud, or behind a gateway it works as on most models
Starting a subagent	the parent's cache is untouched; the subagent builds a cache of its own

One-turn keywords

- **`ultrathink`** anywhere in a prompt asks for deeper reasoning on that turn only, without changing the session's effort; typed in the terminal it can also raise that turn to `xhigh` (the word then shows in rainbow colours). "think", "think hard" and "think more" are ordinary words, not keywords.
- **`ultracode`** in a prompt runs that one task as a workflow without turning the setting on for the session.

▼ **Sources, checked 2026-11-12**

Claim	Source
Levels, defaults, how to set them, `ultracode` as a setting, `ultrathink`, `opusplan`, subagent effort	Model configuration
Subagent model order, `model` and `effort` frontmatter, `CLAUDE_CODE_SUBAGENT_MODEL(_FORCE)`	Subagents
What a model or effort change does to the cache; subagents and the cache	Prompt caching
When a workflow fits, its token cost, the `ultracode` keyword	Dynamic workflows

Version numbers and plan details change; read the pages before relying on a number here.

Suggest a change

Noticed something? Open an issue or a pull request against `main`, changing this page; a steward reads it and a person with the role merges, with a merge commit, so your name stays on it. An AI helper may do all of that up to the merge, signed as the attribution spec says. Not on GitHub? Tell any steward you know, or your helper, in your own words.

Plan limits, free-run counts and what a run costs are Anthropic's to set and change; check their own pages before relying on a number here. This is a description of how the modes behaved when written, not advice on what to spend. Part of the walkthroughs. Icons: Phosphor duotone (MIT).